

Documentación Técnica

Este documento recoge el conjunto de documentación técnica, funcional y de pruebas del sistema Tienda Demo, aplicación de comercio electrónico construida sobre MedusaJS v2. La documentación detalla la arquitectura general, los módulos desarrollados, las reglas de negocio, el diseño de pruebas y las recomendaciones técnicas para futuras versiones del proyecto.

1. Descripción General del Sistema

Tienda Demo es un proyecto de comercio electrónico que tiene como objetivo simular un entorno real de ventas por Internet. El sistema ha sido pensado para servir como base de pruebas de software orientadas a validar reglas de negocio críticas de inventario, descuentos y cálculo de envíos. La solución emplea una arquitectura modular basada en MedusaJS, que permite separar la lógica de negocio en componentes independientes y de fácil mantenimiento. El sistema ha sido diseñado principalmente para facilitar pruebas en un entorno educativo, en lugar de ser utilizado por equipos de calidad (QA), lo que permite la implementación de pruebas unitarias, pruebas de integración y validaciones funcionales en diversos escenarios.

2. Objetivos del Proyecto

El objetivo principal del proyecto es proporcionar una aplicación de demostración que permita validar procesos fundamentales de un sistema e-commerce moderno. Entre los objetivos específicos se encuentran:

- Probar un flujo de compra completo.
- Verificar la disponibilidad del inventario antes de procesar las órdenes.
- Aplicar descuentos automáticos y promociones.
- Calcule los gastos de envío en función de la distancia.
- Construir una base sólida para poder hacer pruebas.
- Hacer más fácil el aprendizaje de la arquitectura modular en MedusaJS.

3. Tecnologías Utilizadas

La solución fue desarrollada utilizando tecnologías modernas orientadas al desarrollo web y backend empresarial.

Backend: Node.js con MedusaJS v2.13.6.

Frontend: React y Vite para la tienda demostrativa.

Base de datos: SQLite.

Testing: Jest y utilidades de testing de MedusaJS.

Control de versiones: Git.

Arquitectura: Modular basada en servicios y workflows.

4. Arquitectura del Sistema

El sistema está organizado en arquitectura de varias capas, donde la capa de presentación se encarga de exponer la API REST y el panel de control, mientras que la capa de lógica de negocio concentra las reglas críticas de inventario, descuentos y envíos. La arquitectura modular facilita el mantenimiento del código, la reutilización de componentes y la realización de pruebas aisladas. Cada módulo tiene sus propios servicios, modelos y pruebas unitarias, lo cual reduce el acoplamiento entre los componentes.

5. Módulo de Inventario

A continuación se muestra una tabla detallada de los principales casos de prueba asociados al módulo inventario:

El módulo de inventario controla la disponibilidad de los productos en la tienda. Su responsabilidad clave es verificar que haya suficiente stock antes de permitir la creación de una orden. En este módulo se realizan comprobaciones sobre cantidades erróneas, productos que no existen y límites de stock. Además, asegura que si alguno de los productos solicitados no cumple con las condiciones de disponibilidad, se rechace la orden completa. La implementación está pensada para permitir pruebas de caja blanca y caja negra, incluidas las comprobaciones de fronteras y particiones de equivalencia.

Escenario de Prueba	Datos de Entrada	Resultado Esperado	Notas y Condiciones
Compra con stock suficiente	Producto con stock=10, cantidad=5	Validación exitosa	Caso válido estándar
Compra con stock insuficiente	Stock=10, cantidad=15	Error INSUFFICIENT_STOCK	Debe rechazarse la orden
Compra con cantidad cero	Cantidad=0	Error INVALID_QTY	Validación de límite inferior
Compra con cantidad negativa	Cantidad=-3	Error INVALID_QTY	No se aceptan cantidades negativas
Producto inexistente	ID producto inválido	Error NOT_FOUND	Producto no registrado

Compra múltiple parcial	Un producto válido y otro inválido	Orden rechazada	Debe invalidarse toda la orden
-------------------------	------------------------------------	-----------------	--------------------------------

6. Módulo de Descuentos

Los casos de prueba siguientes permiten validar el comportamiento del módulo de descuentos:

El módulo de descuentos permite aplicar promociones automáticas o descuentos con códigos promocionales. La lógica puesta en marcha impide que se acumulen descuentos y da prioridad a los códigos promocionales por encima de los descuentos automáticos. El sistema cuenta con validaciones de existencia del código, límite de usos y manejo case-insensitive. Estas reglas garantizan que el cálculo de precios se haga de forma consistente y reducen los errores que se producen con las promociones.

Escenario de Prueba	Condiciones / Entrada	Resultado Esperado	Observaciones
Compra menor a \$200.000	Subtotal=150000	Sin descuento	No supera el umbral
Compra mayor a \$200.000	Subtotal=250000	10% de descuento	Descuento automático
Código promocional válido	Código=DESCUENTO10	Aplicación correcta del descuento	Código existente
Código inválido	Código=INVALIDO	Error INVALID_CODE	Debe notificarse al usuario
Código en minúsculas	descuento10	Código reconocido	Validación case-insensitive
Código + descuento automático	Subtotal=300000 y código válido	Solo aplica el código	No acumulabilidad

7. Módulo de Envíos

La siguiente tabla resume los escenarios principales de validación para el módulo de envíos:

El módulo de envíos calcula el costo de entrega utilizando una fórmula de distancia euclidiana desde el punto de origen de la tienda. Dependiendo de la distancia, el sistema determina si el envío puede realizarse y cuál será el costo asociado.

Este módulo permite validar tanto coordenadas directas como zonas predefinidas almacenadas en la base de datos. También implementa restricciones para evitar compras fuera del rango máximo permitido.

Escenario de Prueba	Datos de Entrada	Resultado Esperado	Condiciones Especiales
Zona cercana	Coordenadas (3,2)	Costo \$5.000	Envío Express
Zona media	Coordenadas (6,4)	Costo \$10.000	Envío Estándar
Zona fuera de rango	Coordenadas (10,8)	Compra rechazada	Distancia > 10 km
Zona inexistente	Zona=NOEXISTE	Error ZONE_NOT_FOUND	Zona no registrada
Coordenadas inválidas	Valores nulos	Error MISSING_LOCATION	Debe indicarse ubicación

8. Roles y Actores del Sistema

El sistema identifica varios actores principales:

Cliente Final: Navega productos, agrega artículos al carrito y realiza compras.

Administrador: Gestiona productos, promociones y zonas de envío.

Sistema: Ejecuta validaciones automáticas y procesa órdenes.

Equipo QA/Desarrollo: Implementa pruebas y garantiza calidad del software.

Cada actor interactúa con el sistema desde diferentes perspectivas funcionales y técnicas, permitiendo una separación clara de responsabilidades.

9. Casos de Prueba

El proyecto implementa un conjunto estructurado de casos de prueba enfocados en las reglas de negocio principales. Actualmente se identifican 29 pruebas entre unitarias e integración.

Las pruebas unitarias validan funciones individuales de cada módulo, mientras que las pruebas de integración verifican el flujo completo de creación de órdenes. Se aplican técnicas como partición de equivalencia, análisis de valores límite y cobertura de condiciones múltiples.

Las pruebas aseguran que el sistema responda correctamente frente a entradas válidas e inválidas, garantizando estabilidad y confiabilidad.

10. Estrategia de Testing

Para complementar la estrategia de testing, se presenta la siguiente clasificación de pruebas implementadas:

La estrategia de pruebas contempla distintos niveles de validación. Las pruebas unitarias verifican el comportamiento interno de los módulos, mientras que las pruebas de integración validan la comunicación entre componentes.

El proceso incluye criterios de entrada y salida claramente definidos, métricas de cobertura y estándares mínimos de calidad. Asimismo, se establecen métricas como porcentaje de éxito de pruebas, cobertura de código y estabilidad del build.

Tipo de Prueba	Elemento Evaluado	Objetivo	Notas
Pruebas Unitarias	Funciones individuales	Validar comportamiento interno	Uso de Jest
Pruebas de Integración	Flujos completos	Validar interacción entre módulos	Incluye creación de órdenes
Pruebas de Caja Negra	Entradas y salidas	Verificar reglas funcionales	Sin conocer implementación
Pruebas de Caja Blanca	Cobertura de código	Validar ramas y condiciones	Incluye valores límite
Pruebas de API	Endpoints REST	Validar respuestas HTTP	Incluye códigos de estado

11. Consideraciones de Calidad

La calidad del software se evalúa mediante cobertura de pruebas, manejo adecuado de errores y cumplimiento de reglas de negocio. El uso de una arquitectura modular permite aislar defectos y mejorar la mantenibilidad del sistema.

Se recomienda implementar pipelines de integración continua, automatización de despliegues y pruebas de rendimiento en futuras versiones del proyecto.

12. Conclusiones

La aplicación Tienda Demo constituye una base sólida para el desarrollo y validación de sistemas de comercio electrónico orientados a pruebas de software. La separación modular facilita la escalabilidad y el mantenimiento del proyecto, mientras que las reglas de negocio implementadas representan escenarios reales comunes en plataformas e-commerce.

La documentación elaborada permite comprender la estructura técnica del sistema, sus funcionalidades principales y la estrategia de calidad aplicada durante el proceso de desarrollo.

13. Recomendaciones Futuras

- Implementar pruebas automatizadas de rendimiento y estrés.
- Integrar pipelines CI/CD utilizando GitHub Actions.
- Migrar SQLite hacia PostgreSQL en ambientes productivos.
- Añadir autenticación y autorización avanzada.
- Expandir la cobertura de pruebas del frontend.
- Implementar monitoreo y trazabilidad de errores.